

1. IMPORTS (TOOLS WE NEED)

```
import json
import boto3
import base64
import uuid
```

What each does:

- json → handles API input/output (REST API communication)
- boto3 → AWS SDK (talks to S3, Rekognition, DynamoDB)
- base64 → converts image from frontend format → binary
- uuid → generates unique ID for each visitor image

Think:

These are our **building blocks for cloud interaction**

2. AWS SERVICE CLIENTS (CONNECT TO AWS)

```
s3 = boto3.client('s3', region_name='eu-west-1')
rekognition = boto3.client('rekognition', region_name='eu-west-1')
dynamodb = boto3.resource('dynamodb', region_name='eu-west-1')
```

What's happening:

- Connecting our Lambda to:
 - S3 → storage
 - Rekognition → face recognition
 - DynamoDB → user database

Important:

Everything is tied to **eu-west-1 (Ireland region)**

3. CONFIGURATION (OUR SYSTEM SETTINGS)

```
BUCKET_NAME = 'samuel-visitor-images'
COLLECTION_ID = 'employees'
TABLE_NAME = 'employee'
```

Meaning:

- BUCKET_NAME → where visitor images are stored
- COLLECTION_ID → Rekognition face database
- TABLE_NAME → DynamoDB table for user info

This is our **system environment config**

4. MAIN FUNCTION (ENTRY POINT)

```
def lambda_handler(event, context):
```

This is where EVERYTHING starts

- Triggered by **API Gateway (REST API)**
- Receives request from frontend

5. GET INPUT FROM API

```
body = json.loads(event.get('body', '{}'))
image_base64 = body.get('image')
```

What's happening:

- API sends JSON:

```
{
  "image": "base64_string_here"
}
```

- We extract the image

ERROR CHECK

```
if not image_base64:
    return response(400, "Missing image")
```

If no image → return error

✓ Good practice: input validation

6. CONVERT IMAGE

```
image_bytes = base64.b64decode(image_base64)
```

Converts:

- Base64 string → actual image file

7. GENERATE UNIQUE IMAGE NAME

```
image_id = str(uuid.uuid4())  
image_key = f"visitors/{image_id}.jpeg"
```

Example output:

visitors/ abc123-xyz.jpeg

- ✓ Prevents overwriting
- ✓ Keeps images unique

8. UPLOAD IMAGE TO S3

```
s3.put_object(  
    Bucket=BUCKET_NAME,  
    Key=image_key,  
    Body=image_bytes,  
    ContentType='image/jpeg'  
)
```

This stores visitor image in S3

- ✓ Now Rekognition can access it

9. FACIAL RECOGNITION (CORE LOGIC)

```
rek_response = rekognition.search_faces_by_image(  

```

What it does:

- Takes uploaded image
- Compares against stored employee faces

PARAMETERS:

CollectionId=COLLECTION_ID

Our employee face database

MaxFaces=1

Only return best match

FaceMatchThreshold=85

Confidence level (85%)

Higher = stricter matching

10. CHECK MATCH RESULT

```
matches = rek_response.get('FaceMatches', [])
```

Extract results

NO MATCH

```
if not matches:  
    return response(200, {"Message": "Failure"})
```

Visitor NOT recognised

11. GET FACE ID

```
face_id = matches[0]['Face']['FaceId']
```

This is unique ID from Rekognition

12. QUERY DATABASE (DYNAMODB)

```
table = dynamodb.Table(TABLE_NAME)  
res = table.get_item(Key={'rekognitionid': face_id})
```

Look up user info using FaceId

NOT FOUND IN DB

```
if 'Item' not in res:  
    return response(200, {"Message": "Failure"})
```

Face exists but no user record

13. GET USER DETAILS

```
user = res['Item']
```

Example:

```
{  
  "FirstName": "Larry",  
  "LastName": "Page"  
}
```

14. SUCCESS RESPONSE

```
return response(200, {  
    "Message": "Success",  
    "firstName": user['FirstName'],  
    "lastName": user['LastName']  
})
```

Sends result back to frontend

15. ERROR HANDLING

```
except Exception as e:  
    print(e)  
    return response(500, "AWS error")
```

If anything fails:

- Logs error
- Returns generic message

16. RESPONSE FUNCTION (API FORMAT)

```
def response(status_code, body):
```

Returns:

```
{
  'statusCode': status_code,
  'headers': {...},
  'body': json.dumps(body)
}
```

IMPORTANT HEADERS

'Access-Control-Allow-Origin': '*'

Enables:

- Frontend (React) to call API

✓ This is **CORS**

FULL FLOW

User → React → API Gateway → Lambda
→ Decode Image
→ Upload to S3
→ Rekognition match
→ DynamoDB lookup
→ Return result

SUMMARY

This code implements:

- Serverless backend
- Image processing pipeline
- AI-based authentication
- Secure API communication

“The Lambda function receives a base64-encoded image via API Gateway, decodes and stores it in S3, performs facial recognition using Rekognition against a stored collection, retrieves user metadata from DynamoDB using the matched FaceId, and returns authentication results to the frontend.”